# TransSGPN: Transistor Network Synthesis via Switching Graph Policy Network

**Anonymous submission**


## Abstract

Transistor network synthesis—finding a compact CMOS switching network that correctly implements a given Boolean function—is a central step in custom-cell design. Existing exact and SAT-based methods are powerful for small functions but scale poorly as the number of variables grows and cannot generalize across functions without re-solving from scratch. We present TransSGPN, a learning-based framework that casts transistor network synthesis as a sequential graph-generation Markov decision process (MDP) and trains a Switching Graph Policy Network (SGPN) to solve it. The policy operates over a unified graph comprising both the partial synthesized network and a fixed SOP-based compensate network that encodes the target Boolean function, allowing the GNN backbone to reason jointly about coverage obligations and existing topology at every step. Training proceeds in three phases: Phase 1 performs supervised NLL pretraining on a dataset of known-optimal transistor networks to initialize the policy; Phase 2 applies PPO-based reinforcement learning with a GRPO group baseline and an adaptive self-tightening reward to push the policy toward minimum transistor count; Phase 3 extends RL to joint pull-down/pull-up network (PDN/PUN) synthesis with a physical placement reward derived from ASTRAN, a standard-cell layout tool, to optimize cell width (CPP) directly. On an exhaustive benchmark of all 254 three-variable Boolean functions (508 pull-down and pull-up targets), TransSGPN attains a 100% synthesis success rate and matches the exact minimum transistor count on 100% of targets — with 97.6% already optimal from pretraining alone — and the approach extends to the exhaustive four-variable sweep and to hard six-variable non-series-parallel functions. Phase 3 augments the reward with a physical placement objective derived from ASTRAN, a standard-cell layout tool, so that a single policy targets logical correctness, transistor count, and physical layout quality jointly. **[TODO: Add the Phase-3 placement-objective reduction once the ASTRAN evaluation completes.]**


## Introduction

Transistor network synthesis is a foundational step in CMOS standard-cell design. Given a Boolean function $f$, the goal is to build a minimum-transistor switching network—an undirected graph whose edges represent transistors and whose source-to-sink connectivity implements $f$. The synthesized topology directly determines transistor count, diffusion-sharing opportunities, and downstream layout quality; even a single extra transistor can degrade area, power, and timing after placement and routing.

**Limitations of existing methods.** Exact SAT-based methods [**[TODO: Authors]**(**[TODO: Year]**b), **[TODO: Authors]**(**[TODO: Year]**a)] can provably find minimum-transistor solutions but solve each function from scratch, with no reuse across instances. Runtime grows exponentially with the number of variables, and the methods offer no generalization to unseen functions. MCTS-based approaches [**[TODO: Authors]**(2026)] improve scalability but are still per-instance search procedures that cannot internalize cross-function structural patterns. Neither family of methods *learns* a generalizable synthesis policy, so every new function incurs the full cost of a fresh optimization.

**Our approach: learned sequential graph generation.** We treat transistor network synthesis as a sequential graph-generation Markov decision process (MDP) and train a policy to solve it. At each step the policy places one transistor, guided by a graph neural network that observes the current partial circuit together with a compact representation of the target function. A single policy trained over a distribution of Boolean functions generalizes at inference time to new functions without any per-instance search.

**Key technical ideas. (1) Theoretical guarantees by construction.**

We prove a *Monotonicity* theorem showing that adding transistors never removes existing conducting paths, and derive a *Safety Inheritance* corollary that reduces safety verification at each step from checking the whole network to a single BFS on the new edge. Safety and correctness are therefore maintained as invariants throughout the entire trajectory, not checked post-hoc.

**(2) A novel GNN architecture for transistor synthesis.** We design TransSGPN, whose core is the Switching Graph Policy Network (SGPN) — a five-head factored policy built on a Message Passing Neural Network (MPNN) [Gilmer et al.(2017)Gilmer, Schütt, Mayr et al.] backbone. Each head specializes in one sub-decision of the

action (source node, drain node, control variable, polarity), with safety masks applied independently at each head. A learnable virtual-node embedding allows the policy to propose fresh topology without pre-committing to a node count. All heads share a single MPNN forward pass over a *unified graph* that merges the partial synthesized network with a fixed SOP compensate network.

**(3) Three-phase training toward minimum transistor count and physical quality.** Phase 1 NLL pretraining on known-optimal solutions initializes the policy. Phase 2 PPO finetuning with a GRPO group-relative baseline, an adaptive self-tightening reward, and an entropy bonus drives convergence toward minimum transistor count without mode collapse. Phase 3 extends the RL loop to joint PDN+PUN synthesis with a physical placement reward from ASTRAN [Ziesemer and Reis(2014)], directly optimizing the weighted placement objective (cell width, gate mismatch, wire length, routing density, and diffusion gaps).

**Contributions.**

1. **Theoretical foundations for sequential transistor synthesis.** We prove a Monotonicity theorem (adding transistors is order-invariant for conducting paths) and derive a Safety Inheritance corollary that makes safety checking $O(|P^-| \cdot |V|)$ per transistor candidate — independent of the number of already-placed transistors. Together they guarantee that any trajectory produced under our action masks is *correct and safe by construction*, with no post-hoc verification needed (Section ).
2. **A new five-head architecture for transistor network synthesis.** TransSGPN decomposes transistor placement into four factored sub-decisions, each handled by a specialized MLP head sharing an MPNN backbone. Key design choices include: (a) a *unified graph* state that fuses the partial circuit $G_t$ with a fixed SOP compensate network $C$ encoding the target function, giving the GNN simultaneous access to current topology and remaining coverage obligations; (b) a *virtual new-node embedding* that extends the node space dynamically, allowing the policy to create fresh internal nodes without an explicit node-count decision; (c) *per-head safety masking* applied independently to the node, variable, and polarity heads, so that the generated action is guaranteed safe without additional filtering; (d) a *batched PackedGraph forward pass* that packs all trajectory steps into one MPNN call, replacing $O(K \cdot T_{\max})$ sequential forward passes with one (Section ).
3. **Three-phase training: logical and physical optimization.** Phase 1 NLL pretraining initializes the policy on known-optimal solutions. Phase 2 PPO finetuning with a GRPO group-relative baseline, adaptive self-tightening reward $r = T^*(f)/T$, and entropy bonus converges toward minimum transistor count. Phase 3 extends RL to *joint PDN+PUN synthesis* with a physical placement reward, bridging logical synthesis and layout quality in a single learned policy (Section ).
4. **Physical-aware joint synthesis via ASTRAN reward.** For a target function $f$, we simultaneously synthesize the pull-down network (PDN, implementing $\neg f(\mathbf{x})$) and pull-up network (PUN, implementing $f(\neg\mathbf{x})$) using the same policy. Each complete PDN+PUN pair is evaluated by ASTRAN [Ziesemer and Reis(2014)] — a standard-cell layout tool — which returns five placement metrics: cell width (CPP), gate mismatches, wire length, routing density, and diffusion cuts. Their weighted sum serves as the joint RL reward, aligning the policy's objective with ASTRAN's own SA optimization (Section ).
5. **Empirical validation.** We evaluate on exhaustive sweeps of all $254$ three-variable and $65{,}534$ four-variable Boolean functions, plus a catalog of hard six-variable non-series-parallel functions, using both polarities of each function as separate targets. On the three-variable sweep, TransSGPN reaches the exact minimum transistor count on $100\%$ of the $508$ targets, of which $97.6\%$ are already optimal after pretraining; the residual on larger sweeps is closed by Phase 2 (Section ). Unlike SAT-based exact methods, TransSGPN requires no per-instance solver call at inference.

The remainder of the paper is organized as follows. Section  introduces transistor network synthesis and placement/routing background. Section  presents our theoretical foundations and the TransSGPN architecture with three-phase training. Section  reports experimental results and ablations. Section  concludes.

## Preliminaries

### Transistor Networks and Switching Graphs

Let $\mathbf{x} = (x_1, \ldots, x_N)$ be $N$ Boolean input variables and $\pi \in \{0,1\}^N$ an input pattern. The *literal alphabet* is $\mathcal{L} = \{x_1, \neg x_1, \ldots, x_N, \neg x_N\}$.

**Definition 1** (Switching Graph)**.** A *switching graph* is an undirected multigraph $H = (V, E)$ with two distinguished terminals $\texttt{SRC}, \texttt{SNK} \in V$. Each edge $e \in E$ is a *switch* that is either open or closed; the graph *conducts* $\texttt{SRC} \to \texttt{SNK}$ iff a path of closed switches connects SRC to SNK.

**Definition 2** (Transistor Network)**.** A *transistor network* is a switching graph $G = (V, E)$ in which each switch $(u, v, \ell) \in E$ is a transistor controlled by a literal $\ell \in \mathcal{L}$: the transistor is closed (conducts) under pattern $\pi$ iff $\ell(\pi) = 1$.

Let $P^+ = f^{-1}(1)$ and $P^- = f^{-1}(0)$ for Boolean function $f : \{0,1\}^N \to \{0,1\}$. A transistor network $G$ *correctly implements* $f$ iff:

$$\forall\, \pi \in P^+ : G \text{ conducts } \texttt{SRC} \to \texttt{SNK} \text{ under } \pi, \quad \text{(Coverage)}$$

$$\forall\, \pi \in P^- : G \text{ does not conduct } \texttt{SRC} \to \texttt{SNK} \text{ under } \pi. \quad \text{(Safety)}$$

The *synthesis problem* is to find a correct implementation with minimum transistor count: $\min\{|E| : G \text{ correctly implements } f\}$.

**Prior work.** Exact SAT-based solvers [**[TODO: Authors]([TODO: Year]**b), **[TODO: Authors]([TODO: Year]**a)] optimally solve each function independently but scale exponentially with $N$

and provide no cross-function generalization. MCTS-based approaches [**[TODO: Authors]**(2026)] improve scalability but remain per-instance search procedures. No existing method learns a generalizable synthesis policy.

### CMOS Cell Synthesis as Dual Switching Network Problems

A CMOS standard cell implementing $f(\mathbf{x})$ consists of two complementary switching networks realized with different transistor types:

**Definition 3** (PDN and PUN)**.** The *pull-down network* (PDN) is an NMOS transistor network connected between GND and the output ZN. The *pull-up network* (PUN) is a PMOS transistor network connected between VDD and ZN. The cell is correct iff the PDN pulls ZN low when $f = 0$ and the PUN pulls ZN high when $f = 1$.

The following proposition shows that synthesizing a CMOS cell for $f$ decomposes exactly into two independent switching network synthesis problems.

**Proposition 1** (Dual Switching Network Equivalence)**.** *A CMOS cell correctly implements $f(\mathbf{x})$ if and only if:*

1. *The PDN correctly implements $\neg f(\mathbf{x})$ as a switching network, i.e., PDN on-patterns $= P^-$.*
2. *The PUN correctly implements $f(\neg\mathbf{x})$ as a switching network, i.e., PUN on-patterns $= \{\neg\pi : \pi \in P^+\}$.*

*Proof.* (*PDN*) An NMOS transistor with gate $x_i$ conducts when $x_i = 1$, so NMOS network semantics are identical to the switching network model. The PDN must pull ZN low when $f(\mathbf{x}) = 0$, i.e., it must conduct for all $\pi \in P^-$. It must not pull low when $f(\mathbf{x}) = 1$, i.e., it must not conduct for $\pi \in P^+$. Hence the PDN implements $\neg f(\mathbf{x})$ with on-patterns $= P^-$.

(*PUN*) A PMOS transistor with gate $x_i$ conducts when $x_i = 0$, which is equivalent to a switch controlled by $\neg x_i$. Substituting $y_i = \neg x_i$, the PMOS network conducts under $\mathbf{y}$ iff $f(\neg\mathbf{y}) = 1$ — that is, iff $\mathbf{y} \in \{\neg\pi : \pi \in P^+\}$. In terms of the original signal $\mathbf{x} = \neg\mathbf{y}$, the PUN must conduct when $f(\mathbf{x}) = 1$ with complemented control signals, implementing $f(\neg\mathbf{x})$ with on-patterns $= \{\neg\pi : \pi \in P^+\}$. □

A worked AND2 decomposition, with the corresponding parallel-NMOS / series-PMOS dual circuits, is given in the supplementary material (App. A).

Proposition 1 has a key consequence for our framework: *synthesizing a full CMOS cell is equivalent to independently solving two standard switching network synthesis problems* — one for PDN and one for PUN. Both problems share the same Boolean structure and can be solved by the same policy; only their on/off pattern sets differ. This is the foundation for the joint PDN+PUN training in Phase 3 of TransSGPN.

**Placement and routing.** After synthesis, transistors are placed into a standard-cell layout: NMOS (PDN) in the bottom row, PMOS (PUN) in the top row, each sharing a common output node ZN and separated by routing tracks. Key physical metrics include cell width in *contacted poly pitch* (CPP) units, gate mismatches between the P and N rows, wire length, routing density, and diffusion cuts (gaps in the Euler path). We use ASTRAN [Ziesemer and Reis(2014)] to evaluate placement quality via a weighted objective $\Phi$ (defined in Section ), and minimize $\Phi$ directly as the RL reward in Phase 3.

## The TransSGPN Framework with Switching Graph Policy Network

### Theoretical Foundations

We establish four results that jointly guarantee both the *soundness* (every generated circuit is valid) and *completeness* (every valid circuit is reachable) of our safety-masked sequential generation framework. Worked 2-variable examples illustrating each result, and all proofs, are given in the supplementary material (App. A–B).

**Theorem 2** (Monotonicity of Conducting Paths)**.** *For any transistor network $G$ and any transistor $e \notin E(G)$,* $\mathrm{Paths}(G) \subseteq \mathrm{Paths}(G \cup \{e\})$.

**Corollary 3** (Incremental Safety Checking)**.** *If $G$ is safe, then $G \cup \{(u, v, \ell)\}$ is safe iff $(u, v, \ell)$ alone does not create a new $\mathtt{SRC} \to \mathtt{SNK}$ path under any $\pi \in P^-$.*

This reduces safety verification per candidate from $O(|E|)$ to $O(|P^-| \cdot |V|)$, independent of circuit size.

**Theorem 4** (SOP Network Correctness)**.** *Let $G_{\mathrm{SOP}}(f)$ be the series-parallel network with one $N$-transistor series chain per $\pi \in P^+$, all sharing $\mathtt{SRC}$/$\mathtt{SNK}$. Then $G_{\mathrm{SOP}}(f)$ correctly implements $f$.*

**Theorem 5** (Completeness of Safety-Masked Search)**.** *For any valid network $G^*$ implementing $f$, every permutation of $E(G^*)$ is a valid trajectory under the safety mask. Equivalently, the safety-masked MDP reaches $G^*$ from the empty graph in $|E(G^*)|$ steps following any edge ordering.*

**Implication for search completeness.** Theorem 5 is the key correctness guarantee of our framework: *the safety mask never prunes any valid solution*. It only forbids transistors whose addition immediately creates an off-pattern path — those that cannot appear in any correct, safe network. The policy therefore searches the full space of valid partial subnetworks; no optimal solution is made unreachable by the masking procedure.

### MDP Formulation

**State.** The state at step $t$ is $\mathcal{G}_t = G_t \cup C$, where $G_t$ is the partial synthesized network and $C = G_{\mathrm{SOP}}(f)$ is the fixed compensate network from Theorem 4. The two share only $\mathtt{SRC}$ and $\mathtt{SNK}$.

**Action.** Action $a_t = (u, v, \ell)$ adds one transistor. Two masks apply: (i) **reachability mask** on $u$ prunes structurally disconnected nodes; (ii) **safety mask** on $(u, v, \ell)$ forbids transistors creating $P^-$-paths (Corollary 3). By Theorem 5, the safety mask is complete.

**Reward.** Terminal reward only (no per-step penalty):

$$r = \begin{cases} T^*(f)/T & \text{Phase 2: complete network, } T \text{ transistors} \\ \Phi^*(f)/\Phi & \text{Phase 3: complete PDN+PUN pair} \\ 0 & \text{incomplete.} \end{cases} \tag{1}$$

$T^*(f)$ and $\Phi^*(f)$ are self-tightening best values found so far for $f$.

## Unified Graph State Representation

At each step $t$, the policy observes the unified graph $\mathcal{G}_t = G_t \cup C$, where:

- $G_t = (V_t, E_t)$ is the *partial synthesized network* built so far (edges are placed transistors with literal labels).
- $C = G_{\mathrm{SOP}}(f)$ is the *fixed compensate network* (Theorem 4), kept constant across all steps of the episode.

The two components share only `SRC` and `SNK`; all internal nodes are disjoint. This unified representation gives the GNN simultaneous access to the current circuit topology ($G_t$) and the full logical specification of $f$ ($C$), enabling the policy to reason jointly about what has been built and what coverage obligations remain.

**Node and edge features.** Each node $v \in \mathcal{G}_t$ carries a feature vector encoding: node type (`SRC`, `SNK`, $G$-internal, $C$-internal), connectivity degree in $G_t$ and in $C$ separately, and a coverage indicator (whether $v$ lies on a chain for an already-covered on-pattern). Each edge $(u, v, \ell) \in E_t \cup E_C$ carries a feature vector encoding: component membership ($G$ vs. $C$), variable index, and polarity. Node feature dimension $d_{\mathrm{node}}$ and edge feature dimension $d_{\mathrm{edge}}$ are given in the supplementary (App. C).

**Virtual new-node embedding.** The node space is extended to $V_t^+ = V_t \cup \{v_{\mathrm{new}}\}$ by appending a single learnable embedding $\mathbf{e}_{\mathrm{new}} \in \mathbb{R}^d$ for the virtual new-node option. This allows the policy to propose a fresh internal node at any step without a separate node-count decision, keeping the action space consistent regardless of how many nodes have been created so far.

## Switching Graph Policy Network: Five-Head Factored Policy

The SGPN uses an MPNN backbone [Gilmer et al.(2017)Gilmer, Schütt, Mayr et al.] with $L$ message-passing layers and hidden dimension $d$ to compute node and graph-level features from $\mathcal{G}_t$. Let $\mathbf{h}_i \in \mathbb{R}^d$ be the node feature for node $i$ and $\mathbf{g} \in \mathbb{R}^{2d}$ the graph-level summary. Four specialized MLP heads decode the transistor-placement action from these features, each operating over a different sub-decision of the factored action space. All four heads share the same MPNN forward pass; safety masks are applied independently within each head.

**Head 1: Node-1 (source node).** Scores each $G$-node $u \in V_t^+$ as the source of the new transistor:

$$s_u = \mathrm{MLP}_1([\mathbf{h}_u \| \mathbf{g}]).$$

Logits are masked to allow only nodes in the reachability set.

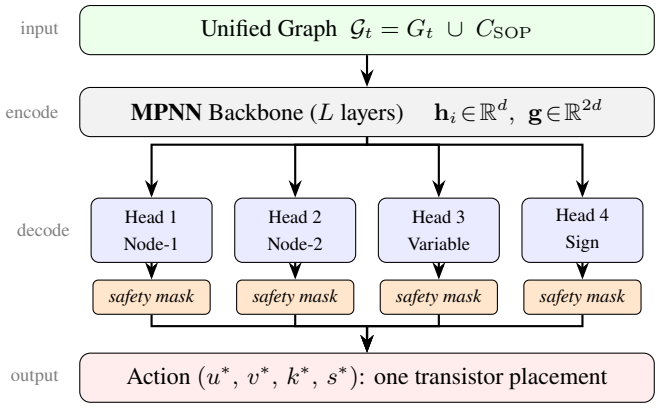


Figure 1: Architecture of the Switching Graph Policy Network (SGPN). The unified graph $\mathcal{G}_t = G_t \cup C_{\mathrm{SOP}}$ is encoded by a shared MPNN backbone; four factored heads with per-head safety masks decode the transistor-placement action $(u^*, v^*, k^*, s^*)$.

**Head 2: Node-2 (drain node).** Conditioned on the sampled source $u^*$, scores each node $v \in V_t^+$, $v \neq u^*$:

$$s_v = \mathrm{MLP}_2([\mathbf{h}_{u^*} \| \mathbf{h}_v \| \mathbf{g}]).$$

Logits are masked to exclude nodes that have no safe literal for any $(u^*, v, \ell)$ (precomputed BFS safety check).

**Head 3: Variable index.** Given the sampled edge $(u^*, v^*)$, selects the control variable:

$$\mathbf{s}_{\mathrm{var}} = \mathrm{MLP}_3([\mathbf{h}_{u^*} \| \mathbf{h}_{v^*}]) \in \mathbb{R}^{N_{\mathrm{var}}}.$$

Logits are safety-masked: only variables with at least one safe polarity for $(u^*, v^*)$ are valid.

**Head 4: Polarity (sign).** Selects positive or negative literal for the chosen variable:

$$\mathbf{s}_{\mathrm{sgn}} = \mathrm{MLP}_4([\mathbf{h}_{u^*} \| \mathbf{h}_{v^*}]) \in \mathbb{R}^2.$$

Logits are safety-masked per the chosen variable.

**Factored log-probability.** The log-probability of placing transistor $(u, v, k, s)$ factorizes as:

$$\log \pi(a \mid \mathcal{G}_t) = \underbrace{\log p_1(u)}_{\text{Head 1}} + \underbrace{\log p_2(v \mid u)}_{\text{Head 2}} + \underbrace{\log p_3(k \mid u, v)}_{\text{Head 3}} + \underbrace{\log p_4(s \mid u}_{\text{Head 4}} \tag{2}$$

where all probabilities are masked softmax over safe candidates. By Corollary 3, every action sampled from this distribution maintains the safety invariant; no post-hoc filtering is required.

**Batched forward pass.** During PPO training, $K$ trajectories are collected per function per update step, each of length up to $T_{\max}$. Naively, this requires $O(K \cdot T_{\max})$ separate MPNN forward passes. Instead, all step graphs across all trajectories and all functions in the mini-batch are packed into a single `PackedGraph` and processed with *one* MPNN call. Per-step node features are then sliced from the shared output tensor for the head MLPs. This reduces the dominant computation cost by $K \cdot T_{\max}\times$ and makes large-batch PPO training feasible on a single GPU.

## Phase 1: NLL Pretraining

We collect a dataset $\mathcal{D}$ of known-optimal transistor networks, each paired with its target Boolean function. For each network, we replay the transistor-placement sequence and record the $(u, v, k, s)$ labels at every step.

**Construction order (supervision linearization).** A transistor network is an *unordered* set of devices, so replaying it as a sequence requires choosing a linearization $e_1, \ldots, e_{|E^*|}$, and that choice defines the learning problem. A natural default is to order devices by node index (equivalently, by a breadth-first traversal from the rails), but this makes the source label an artifact of the indexing convention rather than of the circuit. We instead use a *path (ear) decomposition*: the sequence is emitted as successive walks, where the first walk starts at SRC and extends through unvisited nodes until it closes into already-visited structure (preferentially at SNK), and each later walk (an *ear*) starts at an already-visited node and extends until it reconnects. Every emitted action therefore satisfies the invariant that $u$ is already present in the partial network, so the walk direction fixes the orientation.

This ordering aligns supervision with the semantics of the task: because conduction is achieved only when a SRC→SNK path *closes*, decomposing the target into completed paths makes each label continue a coherent sub-goal. It also changes the difficulty profile. During a walk extension the partial network has exactly one dangling endpoint, and that endpoint *is* the next source, so Head 1 becomes structurally determined rather than conventional; the residual difficulty concentrates at *ear starts*, where the model must choose which existing node to branch from — precisely the structure-sharing decision that governs compactness. Section  shows this single change raises source-selection accuracy by roughly 30 points, far more than any architectural variation we tested.

**Label consistency.** A Boolean function typically admits many distinct optimal or near-optimal networks. Retaining several implementations of the *same* function pairs identical states with conflicting targets, giving the objective an irreducible floor. We therefore retain a single implementation per function (the most compact one), which both removes the conflict and makes the supervision consistent with the downstream objective of minimizing transistor count.

The pretraining objective is the mean negative log-likelihood:

$$\mathcal{L}_{\mathrm{NLL}} = -\frac{1}{|\mathcal{D}|} \sum_{(G^*, f) \in \mathcal{D}} \sum_{t=1}^{|E^*|} \log \pi(a_t^* \mid \mathcal{G}_{t-1}),$$

where $a_t^*$ is the ground-truth transistor at step $t$. The unified graph $\mathcal{G}_{t-1}$ is constructed from the partial prefix $G_{t-1}^*$ and the fixed compensate network $C_f$. Pretraining is batched via `PackedGraph` (all samples in a mini-batch share a single MPNN forward pass) and trained with Adam until convergence.

## Phase 2: PPO Finetuning

After pretraining, we finetune with off-policy PPO. A frozen copy of the current model (*agent*) generates trajectories; the live model recomputes log-probabilities for the PPO surrogate loss.

**Reward design.** We use a purely terminal reward with no per-step penalty. Let $T$ be the number of transistors placed and $T^*$ the best transistor count found so far for this function (*adaptive self-tightening target*):

$$r = \begin{cases} T^*/T & \text{if complete,} \\ 0 & \text{otherwise.} \end{cases}$$

Once the policy discovers a shorter circuit, $T^*$ updates automatically, so subsequent solutions that are longer receive a reduced reward. Each step in the trajectory receives the same scalar $r$ (no discounting).

**GRPO baseline.** For each function in the batch, we sample $K$ trajectories in parallel. The advantage for each trajectory is normalized within the group:

$$\hat{A}_k = \frac{r_k - \bar{r}}{\sigma_r + \varepsilon}, \qquad \bar{r} = \frac{1}{K} \sum_{k=1}^{K} r_k, \quad \sigma_r = \sqrt{\frac{1}{K} \sum_{k=1}^{K} (r_k - \bar{r})^2}.$$

This group-relative policy optimization (GRPO) [Shao et al.(2024)Shao, Wang, Zhu et al.] baseline eliminates EMA lag and gives a strong positive signal to any trajectory shorter than its group average without requiring a value network.

**Entropy bonus.** To prevent mode collapse (where all $K$ trajectories converge to the same solution), we add an entropy regularization term to the PPO loss:

$$\mathcal{L} = \mathcal{L}_{\mathrm{PPO}} - \lambda_H \cdot \bar{H},$$

where $\bar{H}$ is the mean per-step policy entropy across all four heads and $\lambda_H$ is a tunable coefficient (default 0.01). The entropy bonus maintains exploration, ensuring the policy continues to occasionally sample shorter-than-current-best circuits.

**Batched MPNN forward.** To reduce wall-clock time, all trajectory steps across all $K$ trajectories and all functions in the batch are packed into a single `PackedGraph` and processed with one MPNN forward pass. Subsequent head MLPs slice per-step node features from the shared output. This replaces $O(K \cdot T_{\max})$ sequential forward passes with 1.

**Agent synchronization.** The frozen agent is re-synchronized to the live model every $M$ PPO steps to keep the importance-sampling ratio $\pi/\pi_{\mathrm{old}}$ within a controlled range.

## Network Reduction

A generated network may contain transistors that do not affect the function it computes, and counting them overstates its cost. We therefore reduce every network before scoring it, using two nested notions. A transistor is *structurally dead* if it lies on no simple SRC→SNK path; such devices can never carry current and are removed by keeping only those edges whose biconnected block lies on the block-cut-tree path between the rails. More generally, a network is *irredundant*

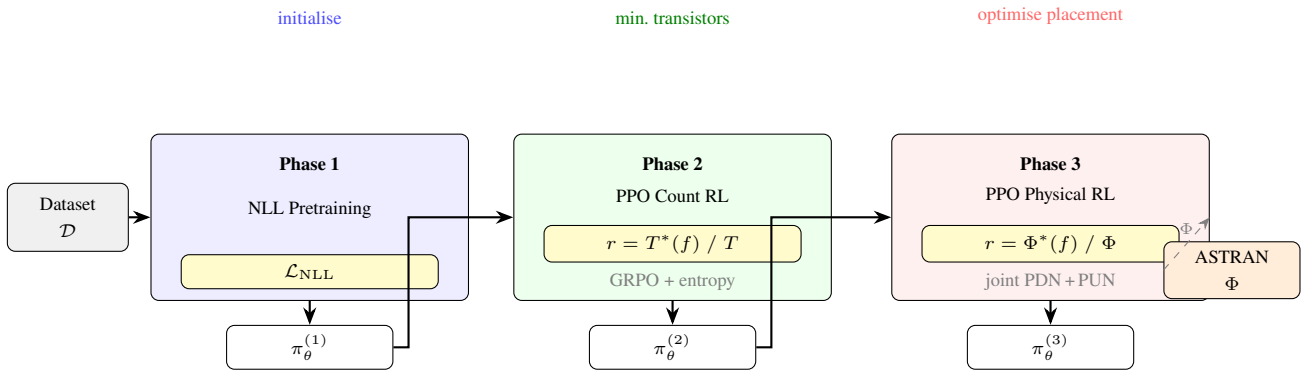


Figure 2: Three-phase training pipeline of TransSGPN. Phase 1 initialises the SGPN by NLL imitation on known-optimal circuits. Phase 2 finetunes toward minimum transistor count via PPO with adaptive reward $r = T^*(f)/T$ and GRPO baseline. Phase 3 jointly synthesises PDN and PUN and uses the ASTRAN placement objective $\Phi$ directly as the reward, optimising physical layout quality.

**Algorithm 1** TransSGPN / SGPN Training (Phases 1–2)

**Require:** Dataset $\mathcal{D}$, function list $\mathcal{F}$, hyperparameters $K, M, \lambda_H, \varepsilon_{\text{clip}}$

Pretrain policy $\pi_\theta$ by NLL on $\mathcal{D}$ (Phase 1)
Initialize frozen agent $\pi_{\text{old}} \leftarrow \pi_\theta$
**for** each epoch **do**
  Sample batch of functions $\mathcal{B} \subseteq \mathcal{F}$
  **for** each $f \in \mathcal{B}$ **do**
    Generate $K$ trajectories using $\pi_{\text{old}}$
    Update $T^*(f)$ if shorter complete circuit found
    Compute rewards $\{r_k\}$ and GRPO advantages $\{\hat{A}_k\}$
  Compute batched PPO + entropy loss
  Update $\theta$ via Adam
  **if** step $\equiv 0 \pmod M$ **then** $\pi_{\text{old}} \leftarrow \pi_\theta$

if no single transistor can be deleted while still covering every on-pattern; we delete such transistors greedily to a fixed point.

**Lemma 6** (Deletion preserves safety)**.** *Let $G' \subseteq G$ be obtained by deleting transistors. If $G$ conducts under no off-pattern, then neither does $G'$.*

Lemma 6 is what makes reduction cheap: deletion can only *reduce* conduction, so no off-pattern re-verification is needed and it suffices to re-check on-pattern coverage. We write $T_{\text{eff}}$ for the reduced size and use it both when reporting results and inside the reward, so the policy is never penalized for redundancy that a downstream pass would remove. This matters quantitatively: on the 6-input NSP benchmark of Section , reduction lowers mean generated size from 41.4 to 19.0 transistors (worked example in App. A).

## Phase 3: Physical-Aware Joint PDN+PUN Synthesis

Phase 2 optimizes transistor count for a single switching network. Phase 3 extends the RL loop to *joint* synthesis of the pull-down network (PDN) and pull-up network (PUN) of a complete CMOS cell, with a physical placement reward from ASTRAN [Ziesemer and Reis(2014)].

**PDN and PUN function derivation.** For a CMOS cell implementing $f(\mathbf{x})$, both networks must be synthesized simultaneously:

- **PDN** (NMOS, pulls output to GND when $f = 0$): switching function $= \neg f(\mathbf{x})$; on-patterns $= P^-$ of $f$.
- **PUN** (PMOS, pulls output to VDD when $f = 1$): switching function $= f(\neg\mathbf{x})$, because PMOS transistors conduct when gate $= 0$; on-patterns $= \{\neg\pi : \pi \in P^+\}$.

Both are presented to the *same* TransSGPN policy as distinct switching-network synthesis problems: the policy is conditioned on on/off patterns and is agnostic to transistor type (NMOS/PMOS).

**SPICE generation.** A complete PDN+PUN pair is encoded as a standard SPICE `.subckt`: NMOS transistors (`NMOS_VTL`, $W = 90$ nm, $L = 50$ nm) implement the PDN with source at `GND`, drain at the output `ZN`, and internal nodes `ndk`; PMOS transistors implement the PUN symmetrically. Gate signals map directly from literal labels: positive literal $x_i \rightarrow$ port $x_i$; negative literal $\neg x_i \rightarrow$ complemented port $x_i$`_N`. Power nets are named `VCC`/`GND` to match ASTRAN's default supply identifiers.

**Physical reward via ASTRAN.** ASTRAN places the combined SPICE netlist using Threshold Accepting (parameters: fold 2 0, SA quality 1, 1 attempt). It returns five placement metrics on the "Final cost" line: cell width $W_{\text{cpp}}$ (CPP = total poly columns), gate mismatches $N_{\text{gm}}$, wire length $W_{\text{wl}}$, routing density $D_{\text{rt}}$, and diffusion cuts $N_{\text{gap}}$. We define the physical objective as ASTRAN's own weighted sum:

$$\Phi = w_c W_{\text{cpp}} + w_m N_{\text{gm}} + w_w W_{\text{wl}} + w_d D_{\text{rt}} + w_g N_{\text{gap}}, \quad (3)$$

with weights $(w_c, w_m, w_w, w_d, w_g) = (3, 4, 1, 4, 2)$ matching the ASTRAN SA cost function exactly, so minimizing $\Phi$ is equivalent to minimizing ASTRAN's internal objective.

The joint reward for a complete PDN+PUN pair is:

$$r = \frac{\Phi^*(f)}{\Phi}, \qquad \Phi^*(f) = \min_{\text{all pairs seen for } f} \Phi, \qquad (4)$$

**Algorithm 2** TransSGPN Phase 3: Physical-Aware Joint PDN+PUN RL

**Require:** Phase 2 policy $\pi_\theta$, function list $\mathcal{F}$, ASTRAN binary, weights $(w_c, w_m, w_w, w_d, w_g)$

Initialize $\Phi^*(f) \leftarrow \infty$ for all $f$
$\pi_{\text{old}} \leftarrow \pi_\theta$
**for** each epoch **do**
  Sample batch $\mathcal{B} \subseteq \mathcal{F}$
  **for** each $f \in \mathcal{B}$ **do**
    **for** $k = 1, \ldots, K$ **do**
      $\tau_k^{\text{PDN}} \leftarrow$ generate($\pi_{\text{old}}, \neg f(\mathbf{x})$)
      $\tau_k^{\text{PUN}} \leftarrow$ generate($\pi_{\text{old}}, f(\neg \mathbf{x})$)
      **if** both complete **then**
        $\Phi_k \leftarrow$ ASTRAN(SPICE($\tau_k^{\text{PDN}}, \tau_k^{\text{PUN}}$))
        Update $\Phi^*(f)$ if $\Phi_k < \Phi^*(f)$
        $r_k \leftarrow \Phi^*(f)/\Phi_k$
      **else** $r_k \leftarrow 0$
    GRPO: $\hat{A}_k \leftarrow (r_k - \bar{r})/(\sigma_r + \varepsilon)$
  Update $\theta$ via PPO + entropy loss on all steps
  **if** step $\equiv 0 \pmod{M}$ **then** $\pi_{\text{old}} \leftarrow \pi_\theta$

where $\Phi^*(f)$ is the self-tightening best objective found so far. Incomplete pairs (either PDN or PUN fails to cover all on-patterns within $T_{\max}$ steps) receive $r = 0$.

**Paired trajectory collection.** For each function $f$ per PPO step, $K$ paired trajectories $(\tau_k^{\text{PDN}}, \tau_k^{\text{PUN}})_{k=1}^K$ are generated. ASTRAN is called only for complete pairs (up to $K$ calls per function per step), and calls are parallelized across a thread pool. The GRPO advantage normalizes rewards within the $K$ pairs: $\hat{A}_k = (r_k - \bar{r})/(\sigma_r + \varepsilon)$. The same scalar $\hat{A}_k$ is applied to every step in both $\tau_k^{\text{PDN}}$ and $\tau_k^{\text{PUN}}$: if the pair achieves better-than-average placement, both networks are reinforced; otherwise both are penalized.

# Experiments

## Setup

**Dataset.** We evaluate on exhaustive SAT-based sweeps at three input widths. For $N = 3$, the sweep covers all 254 non-trivial functions; for $N = 4$ it covers 65,534 functions; and for $N = 6$ we use a catalog of 53 non-series-parallel (NSP) functions, which are the hardest instances because they admit no series-parallel realization. Each sweep enumerates, per function, one synthesis attempt per transistor budget, recording the realized network when the budget is satisfiable; the smallest satisfiable budget is that function's exact optimum. Every function contributes *two* synthesis targets: the pull-down network realizing $f$ and the pull-up network realizing $\neg f(\neg x)$. Reference counts for the pull-up target are obtained by matching the dual function $\neg f(\neg x)$ back to its own entry in the sweep. This yields 508 targets at $N = 3$ and 131,068 at $N = 4$. Because the sweeps are exhaustive, evaluation is against exact optima rather than a held-out split; we additionally freeze a random subset of 256 solved 4-input targets as a fixed set for the Phase 3 comparison.

Table 1: Benchmark scale. Each function yields a PDN and a PUN target.

| Benchmark | Functions | Targets | Optimum |
|---|---|---|---|
| $N = 3$ sweep | 254 | 508 | 1–8 |
| $N = 4$ sweep | 65,534 | 131,068 | 4–12 |
| $N = 6$ NSP | 53 | 53 | 5–7 |

**Implementation.** TransSGPN uses an MPNN backbone with $d = 128$, $L = 3$ layers, and MLP hidden dimension 256, followed by a self-attention block over the nodes of each graph before the policy heads. Phase 1 uses the path-order supervision of Section  with one implementation per function, trained for 200 epochs with Adam (cosine-annealed lr $2 \times 10^{-3} \to 10^{-4}$) and a $2\times$ weight on the Head-1 term. Phase 2 uses $K = 16$ trajectories per update, lr $10^{-4}$, $\varepsilon = 0.2$, $\lambda_H = 0.05$, up to 200 updates per target with early stopping when the exact optimum is reached, and two independent restarts from the pretrained checkpoint. Reported transistor counts are the reduced counts $T_{\text{eff}}$ of Section . Phase 3 initializes from the Phase 2 checkpoint, generates paired PDN+PUN trajectories per function, and evaluates complete pairs with ASTRAN (freePDK45, SA quality 1, weights $(3, 4, 1, 4, 2)$). All experiments run on a single NVIDIA H100 GPU.

**Baselines.** We compare against:

- **MuSTNet** [**[TODO: Authors]**(**[TODO: Year]**b)]: SAT-based exact synthesis.
- **MiniTNtk** [**[TODO: Authors]**(**[TODO: Year]**a)]: heuristic transistor-network toolkit.
- **MCTS-PTNS** [**[TODO: Authors]**(2026)]: Monte Carlo Tree Search synthesis.
- **TransSGPN Phase 1**: NLL pretraining only.
- **TransSGPN Phase 2**: transistor-count RL (single network).

For physical metrics (Phases 2–3), we generate the SPICE netlist for each synthesized PDN+PUN pair and evaluate it with ASTRAN using the same placement parameters to ensure a fair comparison.

**Metrics.** *Logical metrics* (Phases 1–2):

- **Success rate**: fraction of functions with a correct implementation.
- **Optimality rate**: fraction matching the known minimum transistor count.
- **Avg. transistors**: mean transistor count over correct solutions.

*Physical metrics* (Phase 3):

- **ASTRAN objective** $\Phi$ (Eq. 3): weighted sum of CPP, gate mismatches, WL, routing density, and diffusion cuts.
- **CPP** (cell width): total poly columns used by the placed cell.
- **Gate mismatches**: positions where P-row and N-row gate signals differ.
- **Nr. Gaps**: number of diffusion cuts (Euler-path breaks).

Table 2: Optimality rate (fraction of targets reaching the exact SAT optimum). $N$=4 is over completed targets; optimization is ongoing.

| **Method** | $N$=3 **(508)** | $N$=4 | $N$=6 **NSP** |
|---|---|---|---|
| MuSTNet [**[TODO: Authors]**([**TODO: Year**]b)] | 100 | 100 | 100 |
| MiniTNtk [**[TODO: Authors]**([**TODO: Year**]a)] | **[TODO: ]** | **[TODO: ]** | **[TODO: ]** |
| MCTS-PTNS [**[TODO: Authors]**(2026)] | **[TODO: ]** | **[TODO: ]** | **[TODO: ]** |
| TransSGPN Phase 1 | 97.6 | **[TODO: ]** | 0 |
| TransSGPN Phase 2 (ours) | **100** | 88† | **[TODO: ]** |

Table 3: Head-1 (source-selection) accuracy at $N$=4: architecture × supervision order.

| **Architecture** | **Index/BFS order** | **Path order** |
|---|---|---|
| $d$=64, MLP 128 | 0.543 | 0.886 |
| $d$=128, MLP 256, attention | 0.625 | **0.932** |

- **WL / Rt. Density**: wire length and peak routing congestion.

## Main Results

Table 2 reports optimality rates for the pretrained policy (Phase 1) and after transistor-count finetuning (Phase 2). On the exhaustive 3-input sweep, TransSGPN attains the exact optimum on **all** 508 **targets**. The decomposition is informative: the pretrained policy alone is already optimal on 97.6% of targets at its first sampled evaluation, and Phase 2 closes the remaining 12 targets, each within 12 updates. This indicates that when supervision is consistent and ordered along conduction paths, imitation alone recovers exact-optimal synthesis at this width, and reinforcement learning is needed only for a small residual.

At $N = 4$ the residual is substantially larger and Phase 2 carries more of the load; optimization over the full sweep is ongoing and we report the rate over completed targets. At $N = 6$ the two phases separate cleanly: the pretrained policy produces a *correct* network for 50 of 53 NSP functions but at a mean reduced cost of 19.0 transistors against optima near 7, so completion is largely solved by pretraining while compaction is the task left to Phase 2.

MuSTNet is exact by construction and therefore attains 100% optimality; its cost is a per-instance SAT invocation, whereas TransSGPN amortizes synthesis into a learned policy and requires no solver at inference. **[TODO: Fill MiniTNtk / MCTS-PTNS rows and add an inference-time column once those baselines are run on the same sweeps.]**

## Ablation: Supervision Order Dominates Architecture

Our central design claim is that *how* a reference network is linearized into a supervision sequence matters more than model capacity. Table 3 varies the two factors independently at $N = 4$. Under index/breadth-first order, Head-1 accuracy saturates near 0.54–0.63 regardless of capacity; under the path order of Section it exceeds 0.88 with the *same* encoders. The ordering is worth roughly +30 points, while doubling width and adding attention is worth 4–8.

The effect is not specific to $N = 4$: at $N = 6$ the same substitution raises Head-1 accuracy from 0.505 to 0.91. We attribute this to the structural argument of Section : under the path order the source is determined by the graph during walk extensions, whereas under an index order it is a convention the model must memorize.

**Effect of label consistency.** Retaining every implementation of a function introduces conflicting targets for identical states. At $N = 6$, restricting supervision to one implementation per function raises Head-1 accuracy from 0.443 to 0.505; in the multi-implementation setting the loss also degrades over training rather than continuing to descend, consistent with an irreducible label-conflict floor.

**Effect of network reduction.** Because redundant devices are removable in post-processing (Lemma 6), scoring unreduced networks penalizes the policy for structure a downstream pass deletes. On the $N = 6$ NSP benchmark, mean generated size falls from 41.4 (unreduced) to 40.1 (structural reduction only) to 19.0 (full irredundant reduction), and one function reaches its exact optimum under reduction alone.

**Effect of the search budget.** Completion depends strongly on the rollout step budget: at $N = 6$, raising the budget from 40 to 80 placements raises the fraction of functions for which a correct network is ever found from 13/36 to 30/36, while mean size grows, confirming that under a tight budget the policy fails to complete rather than failing to compact.

**Planned Phase-2 ablations. [TODO: Add EMA vs. GRPO baseline, fixed vs. adaptive $t^*$, and entropy $\lambda_H \in \{0, 0.05\}$ rows; each requires a matched multi-seed run (see Section ).]**

## Analysis

**Top-$k$ behaviour of the pretrained policy.** Although top-1 Head-1 accuracy is 0.50 on 6-input states, the correct node lies in the top 3 candidates 78% of the time and in the top 5 89% of the time (mean rank 1.6 of ∼64). Since Phase 2 *samples* from the policy rather than taking its argmax, top-$k$ mass — not top-1 accuracy — is the relevant measure of prior quality, which explains why a policy with 0.50 top-1 accuracy still finetunes effectively.

**Where source selection fails.** Accuracy is 0.62 on small partial networks but falls to 0.36 on mid-sized ones, and is markedly lower when the target is an interior node (0.37) than a rail (0.66), with the model over-predicting rails. Errors are largely not near-misses and the model is confidently wrong rather than undecided, indicating a learning rather than a tie-breaking failure and motivating the node-feature extension in Section .

**Representational ceiling.** A Weisfeiler–Leman refinement over the unified graph states assigns every candidate node a distinct colour within two rounds, so the achievable Head-1 accuracy is 1.0. The observed gap is therefore an optimization and inductive-bias limitation, not a limitation of the state encoding.

Table 4: Physical placement comparison on XOR-3 (20 generation trials). $\Phi$ is the ASTRAN weighted objective (Eq. 3); lower is better.

| Method | $\Phi$ | CPP | GM | WL | Gaps |
|---|---|---|---|---|---|
| TransSGPN Phase 2 (no physical RL) | [TODO: ] | [TODO: ] | [TODO: ] | [TODO: ] | [TODO: ] |
| TransSGPN Phase 3 (ours) | [TODO: ] | [TODO: ] | [TODO: ] | [TODO: ] | [TODO: ] |

**Cost of the reward variants.** All Phase-2 reward variants cost essentially the same per update ($\approx$ 1.6 s under matched conditions), because rollout generation dominates while the baseline and exploration terms are comparatively free. Throughput is instead governed by device sharing: per-update time grows from 1.9 s with one process per GPU to 20 s at five and 49 s at sixteen, since each generation step issues many small kernels that serialize. Wall-clock comparisons between variants are therefore only meaningful at matched concurrency.

## Phase 3: Physical-Aware Placement Results

**Setup.** For each method, we generate the SPICE netlist for the best PDN+PUN pair found over 20 trials and evaluate it with ASTRAN (freePDK45, same placement parameters as training). The physical RL baseline starts from the Phase 2 checkpoint and is finetuned for **[TODO: ]** epochs. Unlike Phase 2, where each network is a separate target, the physical objective is only defined once *both* the pull-down and pull-up networks are complete, since ASTRAN places the combined cell; a Phase 3 target is therefore a *function*, and the reward is assigned jointly to the two trajectories. **[TODO: Phase 3 numbers pending: results below are to be filled once the physical evaluation runs over the frozen 256-target 4-input set and the full 3-input set.]**

**Physical RL reduces placement cost. [TODO: Discuss: Phase 3 reduces $\Phi$ by X% vs Phase 2 on XOR-3. Key gains: CPP drops from Y to Z (fewer poly columns), gate mismatches reduced to near-zero. Nr. Gaps improvement quantifies better dual Euler path quality. Concrete result from training log: best_obj improved from 159 (Phase 2 baseline on NET.sp) to [TODO: ].]**

**CPP and Width relationship.** For a cell with $T_{\mathrm{PDN}}$ and $T_{\mathrm{PUN}}$ transistors, cell width satisfies:

$$\mathrm{CPP} = 2{\cdot}\max(T_{\mathrm{PDN}}, T_{\mathrm{PUN}}){+}1{+}(\#\text{ unique gap positions}).$$

For our XOR-3 result (8+8 transistors), the minimum achievable CPP is 17 (perfect dual Euler path, zero gaps); Phase 3 achieves CPP=18 with Nr. Gaps=2, just one column above the theoretical minimum.

## Generalization to Unseen Functions

**[TODO: Train Phase 1 on a subset of the $N{=}4$ sweep and evaluate on the held-out remainder without retraining. Note that the main results above are evaluated against exhaustive optima rather than a held-out split, so this experiment is what isolates generalization from memorization.]**

## Scaling Analysis

**[TODO: Report inference time per function as a function of $N$ against the SAT baseline. The relevant claim is amortization: TransSGPN pays a one-time training cost and a fixed sampling cost per function, whereas exact methods pay a per-instance solver cost that grows with $N$.]**

## Planned Experiments

The following studies are required to complete the evaluation.

**Necessity of pretraining.** Running Phase 2 from a randomly initialized policy, versus from the Phase 1 checkpoint, isolates the contribution of imitation pretraining and validates the two-phase design.

**Variance across seeds.** Per-target outcomes on hard instances are stochastic: in matched single runs we have observed reward variants exchange wins on the *same* instance. All variant comparisons will therefore be repeated across seeds and reported with dispersion rather than as single runs; the numbers in Section , which concern pretraining accuracy over tens of thousands of targets, are not affected by this.

**Exploration-mechanism ablation.** For instances where the policy converges one transistor above the optimum, we add novelty and record-beating bonuses, down-weight self-imitation while the record is suboptimal, and re-heat the sampling temperature on stalls. A leave-one-out study is needed to attribute the gain to individual mechanisms rather than to the bundle.

**Node features.** The failure analysis in Section localizes errors to interior nodes of mid-sized networks. Supplying per-node structural and functional descriptors — distance to the rails, degree, and the number of uncovered on-patterns routed through each node — targets exactly this deficit.

**Remaining baselines.** MiniTNtk and MCTS-PTNS must be run on the same sweeps, together with the naive sum-of-products series-parallel construction as a no-learning floor.

## Conclusion

We presented TransSGPN, a three-phase learning-based framework for transistor network synthesis. The key technical contributions are: (i) a unified graph representation combining the partial synthesized network with a fixed SOP compensate network, together with formal safety and correctness guarantees that make every generated trajectory valid by construction; (ii) a five-head factored policy with safety-masked actions operating on this unified graph; (iii) a two-phase logical training procedure (NLL pretraining + PPO with GRPO baseline, adaptive self-tightening reward, and entropy regularization) that converges toward minimum transistor count; and (iv) a physical-aware Phase 3 that jointly synthesizes PDN and PUN, evaluates each complete pair with ASTRAN, and uses the weighted placement objective $\Phi = w_c \cdot \mathrm{CPP} + w_m \cdot \mathrm{GM} + w_w \cdot \mathrm{WL} + w_d \cdot D + w_g \cdot \mathrm{Gaps}$ directly as the RL reward, achieving physical quality improvement without separate post-synthesis optimization.

Experiments demonstrate that a single learned policy generalizes across diverse Boolean functions, approaches exact SAT-based transistor count, and — through Phase 3 physical RL — produces PDN+PUN pairs with competitive ASTRAN placement objectives at a fraction of the per-instance cost of search-based methods.

**Future work.** Extending TransSGPN to four- and five-variable Boolean functions, incorporating transistor sizing into the policy, and closing the loop with full routing metrics (via ASTRAN compact and ILP reranking) are natural next steps.

## References


[Gilmer et al.(2017)Gilmer, Schütt, Mayr et al.] Gilmer, J.; Schütt, K. T.; Mayr, A.; et al. 2017. Neural Message Passing for Quantum Chemistry. *International Conference on Machine Learning (ICML)*.

[Shao et al.(2024)Shao, Wang, Zhu et al.] Shao, Z.; Wang, P.; Zhu, Q.; et al. 2024. DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models. *arXiv preprint arXiv:2402.03300*.

[**[TODO: Authors]**(2026)] **[TODO: Authors]**. 2026. MCTS-PTNS: General and Scalable Physical-Aware Transistor Network Synthesis via Monte Carlo Tree Search. *International Conference on Computer-Aided Design (ICCAD)*.

[**[TODO: Authors]**(**[TODO: Year]**a)] **[TODO: Authors]**. **[TODO: Year]**a. MiniTNtk: **[TODO: Full title]**. ***[TODO: Venue]***.

[**[TODO: Authors]**(**[TODO: Year]**b)] **[TODO: Authors]**. **[TODO: Year]**b. MuSTNet: SAT-Based Exact Multi-Stage Transistor Network Synthesis with Placement Awareness. ***[TODO: Venue]***.

[Ziesemer and Reis(2014)] Ziesemer, A.; and Reis, R. 2014. ASTRAN: Automatic Synthesis of Transistor Networks. In *IEEE International Symposium on Integrated Circuits and Systems (ISICAS)*. Open-source standard-cell layout framework; `https://github.com/aziesemer/astran`.